

T.R.
GEBZE TECHNICAL UNIVERSITY
FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING

AI/RAG-BASED LIVE GTU Q&A SYSTEM

ÖMER FEHMI ÇAKICI

SUPERVISOR
MEHMET GÖKTÜRK

GEBZE
2026

T.R.
GEBZE TECHNICAL UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT

AI/RAG-BASED LIVE GTU Q&A SYSTEM

ÖMER FEHMI ÇAKICI

SUPERVISOR
MEHMET GÖKTÜRK

2026
GEBZE

 GEBZE TECHNICAL UNIVERSITY	GRADUATION PROJECT JURY APPROVAL FORM
--	--

This study has been accepted as an Undergraduate Graduation Project in the Department of Computer Engineering on June 2026 by the following jury.

JURY

Member

(Supervisor) : Mehmet Göktürk

ABSTRACT

University candidates, students, and live broadcast audiences often ask institution-specific questions whose answers are distributed across official web pages, regulations, guides, and PDF documents. Generic chatbots can respond fluently, but they may fail to ground answers in the latest institutional context and may not fit the rhythm of a live question-answer setting. This graduation project presents an AI/RAG-based live question-answer system for Gebze Technical University (GTU). The system collects official GTU web pages and PDF documents, archives them locally, splits them into searchable chunks, retrieves relevant context for each user question, and generates concise Turkish answers with a large language model. In addition to a standard web dashboard, the project includes an admin console, YouTube live chat integration, a worker process for queued questions, text-to-speech generation, and a browser-based avatar stage for live presentation.

The backend is implemented with FastAPI, SQLAlchemy, PostgreSQL with pgvector for production vector search, and a SQLite-compatible mode for local tests. The frontend is implemented with Next.js, React, Three.js, and a live-state polling model. Retrieval combines semantic embeddings with keyword, title, URL, document-type, and intent-based scoring so that official GTU documents remain the main evidence source. A 12-question evaluation set was used to measure retrieval and answer behavior. The latest recorded run achieved an 83.33% source hit rate, a 58.33% top-1 source hit rate, a 100% answer keyword hit rate, no fallback answers, and a median latency of 10.07 seconds. These results show that the system is suitable for a controlled live demo while also identifying top-1 ranking precision and latency as the main areas for improvement.

Keywords: retrieval-augmented generation, large language models, live question answering, Gebze Technical University, YouTube live chat, text-to-speech.

ÖZET

Üniversite adayları, öğrenciler ve canlı yayın izleyicileri çoğu zaman cevabı resmi web sayfaları, yönetmelikler, kılavuzlar ve PDF belgeleri içinde dağınık halde bulunan kuruma özgü sorular sorar. Genel amaçlı sohbet sistemleri akıcı cevap üretebilse de, cevapların güncel kurumsal bağlama dayanması ve canlı yayın temposuna uygun olması ayrıca tasarlanmalıdır. Bu bitirme projesinde Gebze Teknik Üniversitesi (GTÜ) için yapay zeka ve RAG tabanlı bir canlı soru-cevap sistemi geliştirilmiştir. Sistem resmi GTÜ web sayfalarını ve PDF belgelerini toplar, yerel arşive kaydeder, içerikleri parçalara ayırır, her soru için ilgili bağlamı geri getirir ve büyük dil modeli ile kısa, anlaşılır Türkçe cevaplar üretir. Standart web arayüzünün yanında admin konsolu, YouTube canlı sohbet entegrasyonu, kuyruktaki soruları işleyen worker süreci, metinden sese üretim ve tarayıcı tabanlı avatar yayın sahnesi de geliştirilmiştir.

Backend tarafı FastAPI, SQLAlchemy, üretim ortamında PostgreSQL ve pgvector, yerel testlerde ise SQLite uyumlu bir yapı ile uygulanmıştır. Frontend tarafında Next.js, React, Three.js ve canlı durum yoklama modeli kullanılmıştır. Geri getirme katmanı anlamsal embedding benzerliğini anahtar kelime, başlık, URL, belge türü ve niyet odaklı puanlarla birleştirerek resmi GTÜ kaynaklarını cevap üretiminin merkezinde tutar. Sistem 12 soruluk bir değerlendirme setiyle ölçülmüştür. Son kaydedilen koşuda kaynak isabet oranı 83.33%, ilk sıradaki kaynak isabet oranı 58.33%, cevap anahtar kelime isabet oranı 100%, fallback oranı 0% ve medyan gecikme 10.07 saniye olarak ölçülmüştür. Sonuçlar sistemin kontrollü canlı demo için kullanılabilir olduğunu, ancak ilk sıra kaynak sıralama başarımı ve gecikme alanlarında iyileştirme gerektirdiğini göstermektedir.

Anahtar Kelimeler: geri getirme destekli üretim, büyük dil modelleri, canlı soru-cevap, Gebze Teknik Üniversitesi, YouTube canlı sohbet, metinden sese.

ACKNOWLEDGEMENT

I would like to thank my advisor Mehmet Göktürk for his guidance during the project, and the Gebze Technical University Computer Engineering Department for providing the academic environment in which this work was developed. I also thank the open-source communities behind FastAPI, Next.js, PostgreSQL, pgvector, Three.js, and the Python data ecosystem, whose tools made it possible to build and evaluate a working end-to-end prototype within the scope of an undergraduate graduation project.

Ömer Fehmi Çakıcı

LIST OF SYMBOLS AND ABBREVIATIONS

Symbol or Abbreviation	:	Explanation
API	:	Application Programming Interface
DB	:	Database
ECTS	:	European Credit Transfer and Accumulation System
GTU	:	Gebze Technical University
HTML	:	HyperText Markup Language
LLM	:	Large Language Model
PDF	:	Portable Document Format
RAG	:	Retrieval-Augmented Generation
TTS	:	Text-to-Speech
UI	:	User Interface

CONTENTS

Abstract	iv
Özet	v
Acknowledgement	vi
List of Symbols and Abbreviations	vii
Contents	ix
List of Figures	x
List of Tables	xi
1 Introduction	1
1.1 Problem Definition	1
1.2 Motivation	1
1.3 Project Objectives	2
1.4 Scope	2
1.5 Contributions	3
1.6 Report Organization	3
2 Background	4
2.1 Retrieval-Augmented Generation	4
2.2 Dense Embeddings and Vector Search	4
2.3 Hybrid Retrieval	4
2.4 Large Language Models for Institutional Answers	5
2.5 Live Question Answering	5
2.6 Technology Stack	5
2.7 Related Work and Documentation	5
3 Methodology	7
3.1 Source Collection Strategy	7
3.2 Archive and Index Pipeline	7
3.3 Chunking and Embeddings	8
3.4 Retrieval Scoring	8

3.5	Answer Generation	8
3.6	Live Processing Pipeline	9
3.7	Evaluation Methodology	9
4	System Design and Implementation	10
4.1	Overall Architecture	10
4.2	Backend API	10
4.3	Database Model	11
4.4	Provider Configuration	12
4.5	Admin Console	12
4.6	Public Dashboard	12
4.7	Live Broadcast Stage	12
4.8	Deployment Topology	13
4.9	Implementation Status	13
5	Experiments and Evaluation	14
5.1	Evaluation Setup	14
5.2	Evaluation Questions	14
5.3	Metrics	14
5.4	Results	15
5.5	Per-Category Observations	15
5.6	Latency Analysis	16
5.7	Failure and Weakness Analysis	16
5.8	Automated Tests	16
5.9	Discussion	17
6	Conclusion and Future Work	18
6.1	Conclusion	18
6.2	Limitations	18
6.3	Future Work	19
6.4	Final Remarks	19
	Bibliography	20
7	Appendices	21
7.1	Local Development Commands	21
7.2	Important API Endpoints	21
7.3	Evaluation Cases	21
7.4	Environment Variables	21

LIST OF FIGURES

4.1	High-level system architecture.	10
4.2	Docker Compose deployment topology.	13

LIST OF TABLES

2.1	Main technologies used in the system.	6
3.1	Source profile used for the GTU corpus.	7
4.1	Core database entities.	11
5.1	Evaluation summary for the 12-question GTU set.	15
7.1	Representative API endpoints.	22
7.2	Evaluation question categories.	22

1. INTRODUCTION

Universities publish important information across many official channels: department pages, candidate portals, academic calendars, regulations, student guides, policy documents, and PDF files. A student or candidate who asks a simple question may therefore need to search several pages before finding an answer. This problem becomes more visible in a live broadcast setting, where questions arrive continuously and answers must be short, understandable, and reliable.

This project addresses that problem for Gebze Technical University (GTU) by developing an AI/RAG-based live question-answer system. The system receives questions from a manual web form or from YouTube live chat, retrieves relevant official GTU context, generates a grounded Turkish answer, and presents the result on a live broadcast screen with optional text-to-speech and avatar animation.

1.1. Problem Definition

Large language models can produce fluent answers, but a university-facing assistant must satisfy stricter requirements than a generic chatbot. It should avoid fabricating institution-specific information, should prefer official sources, should answer in a style suitable for students and candidates, and should keep enough internal trace data for debugging. In a live broadcast scenario, it must also handle incoming questions as a queue and keep the visual state synchronized with speech playback.

The main problem can be summarized as follows:

“ How can official GTU web and PDF sources be transformed into a live, grounded, Turkish question-answer experience that is reliable enough for a controlled broadcast demo? ”

1.2. Motivation

The project was narrowed from a general question-answer tool into a live YouTube broadcast assistant. This scope is motivated by three observations. First, candidate and student questions are often repetitive, but their answers depend on institutional documents that change over time. Second, live streams create a different interaction pattern from a search page: the answer must be concise and must arrive while the audience is still engaged. Third, retrieval traces and admin tooling are necessary because the operator must know whether the system has enough source coverage before

connecting a public stream.

1.3. Project Objectives

The project has the following objectives:

1. Collect official GTU pages and PDF documents from a configurable source profile.
2. Archive raw and parsed sources locally so repeated runs do not depend on re-downloading the same content.
3. Split documents into chunks and index them with embeddings for semantic retrieval.
4. Combine vector retrieval with lexical and intent-based scoring for stronger source precision.
5. Generate Turkish answers that are grounded in retrieved GTU context and avoid exposing internal source traces to viewers.
6. Process questions from both manual input and YouTube live chat.
7. Provide an admin console for ingest, indexing, stream connection, runtime TTS control, and source monitoring.
8. Present the current question and answer on a live stage with avatar, speech, and queue-aware playback state.
9. Evaluate retrieval and answer quality with a repeatable GTU-specific question set.

1.4. Scope

The implemented system is a working prototype for a controlled demo rather than a fully autonomous institutional production service. It supports official GTU web pages, GTU PDF documents, manual questions, YouTube live chat polling, LLM-based answer generation, deterministic fallback behavior, TTS job handling, and a web-based live scene. It does not yet include a large-scale continuous crawling scheduler, authenticated university system integration, human approval workflow, or long-term analytics dashboard.

1.5. Contributions

The main contributions of this project are:

- an end-to-end RAG pipeline specialized for official GTU sources,
- a hybrid retrieval strategy that combines embeddings, keyword matching, title and URL evidence, document-type boosts, and intent-specific bonuses,
- a live question queue that works with manual questions and YouTube live chat messages,
- a broadcast-oriented frontend with avatar state, audio playback, TTS synchronization, and live-state polling,
- a repeatable 12-question evaluation suite with source hit, answer keyword hit, fallback, latency, and confidence metrics.

1.6. Report Organization

Chapter 2 gives background on retrieval-augmented generation, embeddings, vector search, and live question answering. Chapter 3 explains the methodology used for source collection, indexing, retrieval, answer generation, and evaluation. Chapter 4 describes the system design and implementation. Chapter 5 presents the evaluation results and discusses observed limitations. Chapter 6 concludes the report and lists future work.

2. BACKGROUND

This chapter summarizes the technical ideas and tools that support the project. The system combines retrieval-augmented generation, embedding-based search, lexical retrieval signals, web/PDF ingestion, and live web application design.

2.1. Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) connects a generative language model to an external knowledge source. Instead of asking the model to answer only from its parameters, the system first retrieves relevant passages and then sends those passages with the user question to the model. This approach is especially useful for domain-specific information because the system can update its evidence corpus without retraining the model [1].

For the GTU assistant, RAG is necessary because the expected answers include current university pages, regulations, guides, and policy documents. The language model is used for wording and summarization, while the retrieval layer is responsible for finding the official context.

2.2. Dense Embeddings and Vector Search

Dense retrieval represents text as a numerical vector. Related texts are placed close to each other in vector space, so the system can search by semantic similarity rather than exact keyword overlap [2]. In this project, each document chunk receives an embedding. At question time, the query receives an embedding as well, and candidate chunks are ranked partly by cosine similarity.

The production-oriented database design uses PostgreSQL with the pgvector extension so that embeddings can be stored and searched inside the relational database [3]. SQLite is also supported for local tests by storing embeddings as JSON arrays and calculating similarity in application code.

2.3. Hybrid Retrieval

Pure vector search can miss details that are explicit in titles, URLs, document numbers, or regulations. For example, a question about “YN-0001” or “academic calendar” should benefit from exact or near-exact textual evidence. Therefore, the

retrieval layer combines semantic similarity with string relevance, title overlap, URL overlap, phrase matching, document-type boosts, and intent-specific bonuses.

This hybrid design reflects the structure of GTU information. Official regulation PDFs often carry document numbers in their filenames. Department and candidate pages include descriptive URL paths. Academic calendar and ECTS pages are better found when the retrieval logic respects those specific tokens.

2.4. Large Language Models for Institutional Answers

The language model receives a GTU-specific system prompt and a compact set of retrieved contexts. The prompt instructs the model to answer in Turkish, keep the first sentence direct, avoid artificial phrases such as “according to the context”, avoid exposing raw source names or URLs to viewers, and explicitly avoid inventing information when the provided context is weak.

The implementation can use OpenAI directly or OpenRouter through an OpenAI-compatible client. Chat and embedding model names are configurable with environment variables, and deterministic local fallbacks are available when provider keys are missing.

2.5. Live Question Answering

A normal web Q&A application can answer one request at a time. A live broadcast assistant needs additional state. Questions may be pending, processing, answered, or failed. Answers may wait for TTS generation. The visible stage must know whether the avatar should be idle, thinking, speaking, or showing an error. The system therefore separates answer generation from playback and exposes a live-state endpoint that the frontend polls.

2.6. Technology Stack

Table 2.1 summarizes the main technologies used in the project.

2.7. Related Work and Documentation

The project follows the RAG formulation introduced by Lewis et al. [1] and the dense retrieval direction represented by Dense Passage Retrieval [2]. It also relies on official documentation for FastAPI [4], Next.js [5], PostgreSQL [6], pgvector [3],

Table 2.1: Main technologies used in the system.

Layer	Technologies
Backend API	FastAPI, Pydantic, SQLAlchemy, Uvicorn
Retrieval and data	PostgreSQL, pgvector, SQLite test mode, pypdf, BeautifulSoup
AI provider layer	OpenAI-compatible client, OpenAI or OpenRouter configuration
Frontend	Next.js, React, TypeScript, Framer Motion, lucide-react
Avatar and graphics	Three.js, React Three Fiber, Drei, FBX avatar assets
Deployment	Docker Compose, Caddy reverse proxy, Redis placeholder service
Evaluation	Python quality-suite script and JSON report output

YouTube Data API [7], and OpenAI-compatible APIs [8]. These sources shaped the implementation choices but the final behavior is specialized for the GTU live Q&A use case.

3. METHODOLOGY

This chapter explains the approach used to collect official GTU sources, build the retrieval index, generate answers, manage live broadcast state, and evaluate quality.

3.1. Source Collection Strategy

The source profile contains official GTU domains and selected PDF URLs. The seed list prioritizes the main university site, candidate portal, ECTS pages, academic calendar, international pages, department pages, student affairs pages, regulations, and student guides. The crawler can also discover links and PDF files from visited pages, but it only follows URLs that match allowed domains and configured include patterns.

The project intentionally avoids third-party pages, social media content, and forum content. This decision keeps the knowledge base institution-focused and reduces the risk of answering from unofficial information.

Table 3.1: Source profile used for the GTU corpus.

Source group	Examples
Main and candidate pages	<code>www.gtu.edu.tr</code> , <code>aday.gtu.edu.tr</code>
Academic information	ECTS package, academic calendar, student affairs pages
Department pages	Computer, electronics, chemistry, and other GTU department pages
Regulations and guides	Undergraduate and graduate regulations, thesis writing guide, Erasmus directive
Student life	Dormitory, transportation, scholarships, candidate FAQ pages

3.2. Archive and Index Pipeline

The ingestion service uses a two-stage workflow. In the first stage, it crawls or downloads official sources and stores raw files, parsed content, and metadata under a local source archive. In the second stage, it indexes the archived content into the database. This separation makes repeated experiments faster because already downloaded sources can be reused.

HTML pages are parsed with BeautifulSoup. The parser removes common noise

nodes such as navigation, scripts, forms, hidden elements, and social or footer blocks. It then selects the most content-rich node using content selectors, heading and paragraph counts, class/id hints, and link-density penalties. PDF documents are parsed with pypdf, and the extracted text is normalized before chunking.

3.3. Chunking and Embeddings

Documents are split into overlapping chunks. The default configuration uses a chunk size of 400 tokens and an overlap of 80 tokens. Overlap helps preserve context near chunk boundaries. Each chunk receives an embedding through the configured provider. If no provider key is available, a deterministic hashed embedding fallback is used so that local tests can still run.

3.4. Retrieval Scoring

At question time, the system normalizes the query and creates a query embedding. Candidate chunks come from vector search and string-based filters. Each candidate receives a final score calculated from vector similarity, lexical relevance, intent bonuses, and penalties:

$$S = ((0.35V) + (0.55K) + I) \times P \quad (3.1)$$

Here, V is the non-negative cosine similarity between query and chunk embeddings, K is a string relevance score, I is an intent-match bonus, and P is a product of page-kind and document-quality penalties. The exact implementation also considers title overlap, URL overlap, phrase matches, ordered token pairs, document type, and topic hints.

3.5. Answer Generation

The language model receives the user question and at most three retrieved contexts. The system prompt defines the desired assistant behavior: calm, clear, institution-focused, Turkish, concise, and suitable for live broadcast. It also instructs the model not to reveal raw source labels or internal trace data to the viewer.

The model is asked to return JSON with two fields: `display_answer` for the screen and `speech_answer` for TTS. If the provider request fails, the local fallback summarizes retrieved contexts or states that no reliable GTU context is available.

3.6. Live Processing Pipeline

The live system uses a queue-based pipeline. Manual questions can be submitted from the web dashboard, while YouTube live chat messages are collected by the worker through the YouTube Data API. The worker processes active streams, creates new questions, generates answers for pending questions, prepares TTS jobs, and advances the broadcast playback state.

Algorithm 1 Live question processing pipeline.

Require: Official GTU index, optional YouTube stream, pending question queue

Ensure: Updated live state with current question, answer, speech, and avatar phase

while worker is running **do**

 Poll active YouTube streams for new chat messages

 Insert unseen messages as pending questions

 Select the oldest pending question

 Retrieve relevant GTU chunks

 Generate display and speech answers

 Store answer and internal source traces

 Queue TTS job when speech is enabled

 Update broadcast playback state

 Wait for the configured polling interval

end while

3.7. Evaluation Methodology

The evaluation set contains 12 GTU-specific questions. Each case includes expected source hints, expected answer keywords, a category, and notes. The quality suite asks each question through the same RAG service used by the application and records source hits, answer keyword hits, fallback use, latency, confidence, and top source information.

The main metrics are:

- **Top-1 source hit rate:** highest-ranked source matches expected evidence.
- **Source hit rate:** any retrieved trace matches expected evidence.
- **Answer keyword hit rate:** generated answer contains expected key terms.
- **Fallback rate:** deterministic fallback output was used.
- **Latency:** end-to-end answer generation time in milliseconds.

4. SYSTEM DESIGN AND IMPLEMENTATION

This chapter describes the implemented architecture, backend services, frontend views, data model, and deployment structure.

4.1. Overall Architecture

The system is organized as a web application with a backend API, a background worker, a frontend, a database, and a reverse proxy. The backend owns ingestion, retrieval, generation, metrics, stream state, and admin operations. The worker continuously polls YouTube and prepares queued answers. The frontend provides the public dashboard, admin console, and live broadcast stage.

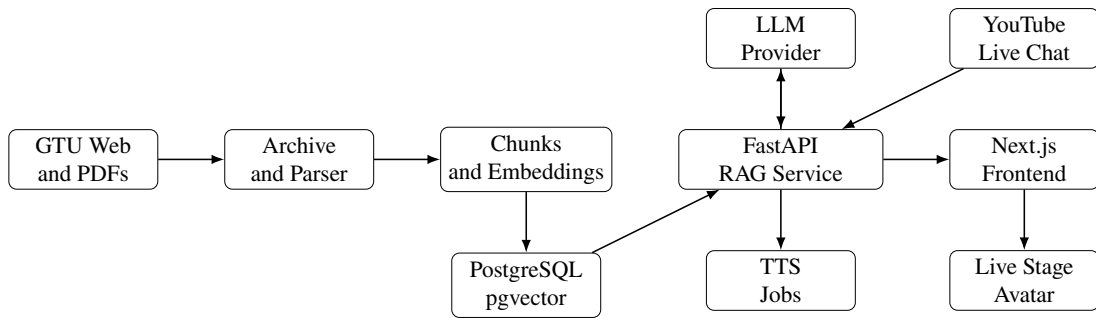


Figure 4.1: High-level system architecture.

4.2. Backend API

The backend is implemented with FastAPI. It initializes the database on startup, mounts generated audio files under a media route, and exposes endpoints under the `/api` prefix. Public endpoints include health checks, manual question submission, question listing, metrics, and live state. Admin endpoints require HTTP Basic authentication and control source ingest, archive crawling, archive indexing, PDF upload, index rebuild, document listing, YouTube stream connection, archive statistics, and runtime settings.

The backend is split into services:

- **IngestService:** crawls sources, parses HTML/PDF content, archives files, chunks text, and stores embeddings.
- **RagService:** retrieves chunks, calls the LLM, and records results.

- **LLMService:** builds prompts, manages provider calls, parses JSON answers, and handles fallbacks.
- **EmbeddingService:** generates provider embeddings or deterministic hashed embeddings.
- **YouTubeService:** connects to live chat sessions and polls new messages.
- **SpeechService and TTSService:** queue, cache, generate, and attach answer audio.
- **BroadcastService:** prepares stage state, ambient segments, playback phases, and avatar states.

4.3. Database Model

The database model is designed around documents, chunks, questions, answers, traces, streams, speech jobs, and broadcast playback. Table 4.1 summarizes the most important entities.

Table 4.1: Core database entities.

Entity	Purpose
Document	Stores source metadata, type, title, URL, filename, checksum, and document-level metadata.
Chunk	Stores chunk content, ordinal position, token count, metadata, and embedding vector.
Question	Stores manual or YouTube questions with status and source information.
Answer	Stores generated answer text, speech text, model name, latency, confidence, fallback flag, and audio metadata.
AnswerTrace	Stores internal evidence snippets and retrieval scores for each generated answer.
StreamSession	Stores connected YouTube stream metadata and polling state.
SpeechJob	Stores queued or generated TTS jobs for answers and ambient broadcast segments.
BroadcastPlayback	Stores the current global playback item and timing state for the live stage.

4.4. Provider Configuration

The system can target OpenAI or OpenRouter through a common OpenAI-compatible client. Provider selection is controlled by `AI_PROVIDER`. Chat and embedding models can be set with `AI_CHAT_MODEL` and `AI_EMBEDDING_MODEL`, while provider-specific defaults remain available. TTS uses a separate provider setting so that the answer model and voice model can be selected independently.

If no API key is configured, embeddings fall back to deterministic hashed vectors and answers fall back to local summarization. This design keeps tests deterministic and prevents local development from failing only because a paid provider key is missing.

4.5. Admin Console

The admin console is implemented as a Next.js page. It allows the operator to load the default GTU source profile, edit seed URLs and PDF URLs, control allowed domains and URL patterns, crawl sources to the local archive, index archived sources, upload PDFs, rebuild indexes, connect a YouTube stream, monitor archive statistics, list indexed documents, and enable or disable TTS.

This console is important because a live assistant should not be treated as a black box. Before a demo, the operator can verify that the corpus has enough sources and that the active stream is connected.

4.6. Public Dashboard

The public dashboard provides a manual question form, recent question feed, and system metrics. Manual questions are placed into the queue rather than being forced immediately onto the stage. This matches the live broadcast behavior: the system can continue the current answer or ambient segment before moving to the next question.

4.7. Live Broadcast Stage

The live stage is a 16:9 broadcast-style page. It polls `/api/live/state`, displays the current question and answer, plays answer audio when available, and drives avatar state. The avatar is loaded with React Three Fiber and Three.js. Mouth movement is influenced by audio duration, playback timing, and a Web Audio meter, so speech animation lasts approximately as long as the spoken answer.

The stage separates visual state from backend processing. A question can be

pending, an answer can be generated, TTS can still be pending, and the previous speech can continue playing. This separation prevents the UI from skipping answers too quickly.

4.8. Deployment Topology

The repository includes a Docker Compose configuration with PostgreSQL, pgvector, Redis, backend, worker, frontend, and Caddy. PostgreSQL is the main production data store. Redis is included for future queue/cache expansion, although the current demo primarily uses database state and the worker loop. Caddy acts as a reverse proxy and can use HTTPS when a production domain is configured.

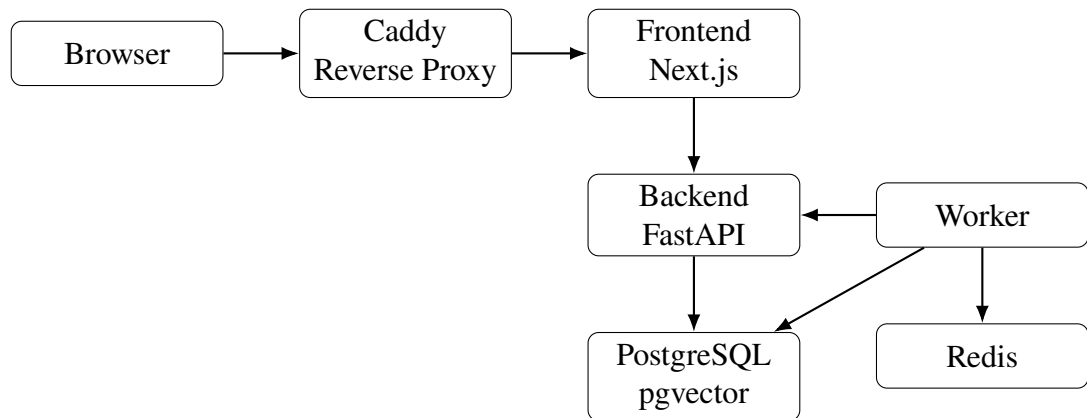


Figure 4.2: Docker Compose deployment topology.

4.9. Implementation Status

The implemented prototype covers the main data path from official GTU sources to archive, index, retrieval, generation, TTS job handling, live state, and visual presentation. It also includes automated tests for API behavior, evaluation metrics, provider selection, LLM response parsing, TTS behavior, and live-stage queue behavior.

5. EXPERIMENTS AND EVALUATION

This chapter presents the evaluation setup, metrics, results, and interpretation. The reported values come from the JSON quality report generated by the project quality-suite script.

5.1. Evaluation Setup

The evaluation uses a GTU-specific question set stored as JSON. Each question includes expected source indicators and expected answer keywords. The cases cover regulations, guides, web pages, department information, and PDF documents. The quality suite submits each question through the same manual-question path used by the application, then inspects the stored answer and retrieval traces.

The recorded run used a local SQLite quality-suite database. This local mode is useful for repeatable development tests. Production deployment can use PostgreSQL with pgvector for vector operations and database extensions.

5.2. Evaluation Questions

The 12 questions were grouped into four categories:

- **Regulations:** undergraduate semester duration, graduate education regulation, thesis writing guide, diploma directive.
- **Web pages:** ECTS package, academic calendar, regulation page, Computer Engineering department description.
- **Department information:** Computer Engineering mission and student profile.
- **PDF documents:** research policy, Erasmus directive, 2025–2026 student guide.

5.3. Metrics

The quality suite calculates five primary metrics. Source hit rate measures whether any retrieved trace matches the expected source hints. Top-1 source hit rate is stricter and only considers the first retrieved trace. Answer keyword hit rate checks whether the generated response includes expected key terms. Fallback rate measures

whether the system used deterministic fallback output instead of the provider model. Latency measures end-to-end answer generation time.

5.4. Results

Table 5.1 shows the summary from the latest recorded evaluation.

Table 5.1: Evaluation summary for the 12-question GTU set.

Metric	Value
Question count	12
Top-1 source hit rate	58.33%
Source hit rate	83.33%
Answer keyword hit rate	100.00%
Fallback rate	0.00%
Average latency	11.93 s
Median latency	10.07 s
P95 latency	18.43 s
Maximum latency	23.34 s
Average confidence	0.5254

5.5. Per-Category Observations

Regulation and directive questions performed well when the expected source was a distinctive PDF. Examples include the undergraduate regulation, thesis writing guide, diploma directive, and Erasmus directive. These questions often contain regulation or guide terms that the retrieval system can match in filenames, titles, or PDF content.

Some web navigation questions were harder. The ECTS and academic calendar questions had weaker top-1 behavior because other official sources contained related terms but were not the most precise answer source. This shows that official but indirect sources can outrank direct pages when the corpus contains broad student guides or regulation documents.

The Computer Engineering department questions performed well because the department page contains distinctive phrases such as the goal of training engineers who can both seek and create jobs. Department-focused questions benefited from title, URL, and phrase overlap.

5.6. Latency Analysis

The median latency was 10.07 seconds and P95 latency was 18.43 seconds. This is acceptable for a controlled demo but high for a polished live production experience. The latency includes retrieval, provider calls, database writes, and answer post-processing. If TTS generation is included in the viewer experience, perceived latency can be higher unless audio generation is decoupled from screen answer display.

The largest observed latency was 23.34 seconds for a web question about the ECTS package. This suggests that navigation-style questions may need better direct-page routing or cached canonical answers.

5.7. Failure and Weakness Analysis

The system did not use fallback output in the recorded run, and all answers matched expected keywords. However, top-1 source precision is the main weakness. When the first source is wrong but a correct source appears lower in the retrieved traces, the generated answer may still be acceptable, but debugging and trust are weaker. For a university-facing assistant, improving first-ranked evidence is important.

The most visible top-1 misses were:

- graduate regulation question: the correct source appeared in traces, but a generic regulation listing page ranked first;
- ECTS package question: an Erasmus directive ranked first, even though ECTS pages were expected;
- academic calendar question: a student handbook PDF ranked first and the answer stated that the direct page was not found;
- research policy question: a different policy-related directive ranked first while the expected research policy document appeared elsewhere.

These misses indicate that the system needs stronger canonical page boosts, better PDF title normalization, and possibly hand-curated route hints for high-value GTU pages such as ECTS and academic calendar.

5.8. Automated Tests

The backend includes automated tests for ingestion and manual question flow, manual queue live-state behavior, live-stage playback timing, ambient segment behavior, evaluation metric calculations, provider option resolution, LLM JSON parsing, and

TTS runtime settings. These tests protect core behavior in the prototype and are especially useful because many production paths can be exercised without a live provider key.

5.9. Discussion

The results show that the project reached the goal of a working controlled demo. The answer coverage is strong, the source hit rate is usable, and the system can operate with real application services rather than isolated scripts. At the same time, the evaluation makes the next engineering priorities clear: improve top-1 retrieval precision, reduce answer latency, expand the official GTU corpus, and add stronger monitoring around weak-context answers.

6. CONCLUSION AND FUTURE WORK

This graduation project developed an AI/RAG-based live question-answer system for Gebze Technical University. The project began with the need to answer GTU-specific questions from official sources and evolved into a live broadcast assistant that can receive manual or YouTube questions, retrieve relevant GTU context, generate Turkish answers, prepare speech, and present the response through a live avatar stage.

6.1. Conclusion

The implemented system satisfies the main project objectives. It can collect and archive official GTU web and PDF sources, index documents as chunks with embeddings, retrieve evidence with a hybrid scoring method, generate institution-focused Turkish answers, keep internal answer traces, manage live question queues, and expose both admin and viewer-facing interfaces. The Docker Compose setup also provides a realistic single-server deployment path.

The evaluation demonstrates that the prototype is usable for a controlled demo. On the 12-question GTU evaluation set, the system reached an 83.33% source hit rate and 100% answer keyword hit rate. The top-1 source hit rate of 58.33% and median latency of 10.07 seconds show that the system works but still needs optimization before it can be considered production-ready.

6.2. Limitations

The main limitations are:

- the corpus depends on the current source profile and may miss newly published or deeply nested pages;
- top-1 retrieval precision is not yet high enough for fully unattended use;
- LLM and TTS provider calls create noticeable latency in live settings;
- the YouTube pipeline depends on API keys, active live chat availability, and polling intervals;
- the avatar animation is prototype-level and does not yet include full phoneme-level lip synchronization;
- there is no human approval workflow for high-stakes institutional answers.

6.3. Future Work

Future work should focus on four areas. First, retrieval can be improved with canonical URL boosts, better source-type routing, stronger document title cleaning, and more complete official GTU coverage. Second, latency can be reduced through answer caching, model selection, background precomputation for common questions, and parallel TTS preparation. Third, the live experience can be improved with richer avatar motion, better audio synchronization, improved queue transitions, and operator moderation controls. Fourth, production readiness can be strengthened with scheduled recrawling, audit logs, dashboard analytics, and human review for sensitive answers.

6.4. Final Remarks

The project shows that a university-specific live assistant can be built by combining RAG, official source archiving, modern web application design, and a broadcast-aware presentation layer. The most important design principle is not simply to make the model answer, but to make the system answer from the right evidence, at the right time, and in a form that fits the live audience.

BIBLIOGRAPHY

- [1] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, 2020. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [2] V. Karpukhin et al., “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020, pp. 6769–6781. [Online]. Available: <https://arxiv.org/abs/2004.04906>
- [3] pgvector. “Pgvector: Open-source vector similarity search for postgresql,” Accessed: Jun. 14, 2026. [Online]. Available: <https://github.com/pgvector/pgvector>
- [4] FastAPI. “Fastapi documentation,” Accessed: Jun. 14, 2026. [Online]. Available: <https://fastapi.tiangolo.com/>
- [5] Vercel. “Next.js documentation,” Accessed: Jun. 14, 2026. [Online]. Available: <https://nextjs.org/docs>
- [6] PostgreSQL Global Development Group. “Postgresql documentation,” Accessed: Jun. 14, 2026. [Online]. Available: <https://www.postgresql.org/docs/>
- [7] Google for Developers. “Youtube data api documentation,” Accessed: Jun. 14, 2026. [Online]. Available: <https://developers.google.com/youtube/v3>
- [8] OpenAI. “Openai api documentation,” Accessed: Jun. 14, 2026. [Online]. Available: <https://platform.openai.com/docs>

APPENDICES

Local Development Commands

The repository includes scripts for both Docker-based and local development. The Docker path starts the complete service stack:

```
cd /Users/omer/Desktop/bitirme  
bash scripts/docker_up.sh
```

For local backend development:

```
cd backend  
python3 -m venv .venv  
source .venv/bin/activate  
pip install -r requirements.txt  
uvicorn app.main:app --reload
```

For local frontend development:

```
cd frontend  
npm install  
npm run dev
```

Important API Endpoints

Evaluation Cases

The quality suite uses 12 questions. Table 7.2 gives a compact overview.

Environment Variables

The main environment variables are:

- `AI_PROVIDER`: selects `openai` or `openrouter`;
- `OPENAI_API_KEY` and `OPENROUTER_API_KEY`: provider keys;

Table 7.1: Representative API endpoints.

Endpoint	Purpose
GET /api/health	Health check.
POST /api/questions/manual	Ask and answer a manual question immediately.
POST /api/questions/manual/queue	Add a manual question to the live queue.
GET /api/live/state	Return current broadcast state for the live stage.
POST /api/admin/archive/crawl	Download and archive official sources.
POST /api/admin/archive/index	Index archived sources into documents and chunks.
GET /api/admin/archive/stats	Return archive statistics.
POST /api/streams/youtube/connect	Connect a YouTube live chat session.

Table 7.2: Evaluation question categories.

Category	Count	Examples
Regulation	4	undergraduate duration, graduate regulation, thesis guide, diploma directive
Web	3	ECTS package, academic calendar, regulation page
Department	2	Computer Engineering mission and target student profile
PDF	3	research policy, Erasmus directive, student guide

- `AI_CHAT_MODEL` and `AI_EMBEDDING_MODEL`: optional model overrides;
- `YOUTUBE_API_KEY`: key used for live chat polling;
- `DATABASE_URL`: database connection string;
- `TTS_ENABLED`: runtime default for speech generation.